Robust Low-Bitrate Neural Speech Coding for Resource-Constrained Communications

Andrew Thibodeau
Computer Science
Virginia Tech
Alexandria VA, USA
tandrew03@vt.edu

Wayne Cleary
Computer Science
Virginia Tech
Alexandria VA, USA
waynercleary@vt.edu

Matthew Hearne
Computer Science
Virginia Tech
Alexandria VA, USA

***Abstract*— Neural speech codecs offer the ability to deliver high-quality speech under a multitude of constraints that emulate real-world conditions. These speech codecs perform efficiently when constrained by compute power, memory, bitrate limits, and latency. Practical deployment on resource-constrained devices such as embedded systems and mobile devices requires simultaneous control of all constraints whilst accounting for acoustic conditions seen in the real world. This project investigates robust low-resource speech in the context of the IEEE LRAC 2.0 Challenge at ICASSP 2027. The project will begin with the provided baseline codec and expand to focus on noise and reverberation-aware training while adhering to the challenge constraints on bitrate, latency, and computation. The proposed solution aims to use a hybrid Linear Predictive Coding (LPC) - neural architecture. LPC analysis will separate the predictable speech from the excitation residual with a lightweight neural encoder and quantization stage will compress the residuals. The objective is to determine whether classical speech modeling can reduce the computational burden of learned components whilst preserving quality and intelligibility across multiple bitrate settings. The final system will be evaluated under the LRAC 2.0 constraints on latency, bitrate, and computational complexity using clean, noisy, reverberant, and multilingual speech. Experiments will compare the proposed LPC-neural architecture with the provided baseline and analyze quality-efficiency tradeoffs across 1, 6, 12, 24, and 32 kbps. Final evaluation will be based on a crowdsourced subjective battery with weights applied to different multilingual, goal, and test conditions.**

***Index Terms*— hybrid architectures, low-bitrate speech coding, neural speech codecs, robust speech coding**

I. Introduction

A. Motivation

There is an increasing need for speech interfaces on devices with limited processing power such as embedded systems and mobile devices, these devices require speech codecs that are efficient under different constraints [1]. Speech coding compresses the original signal into a compressed format for transmission and then is decoded to get the original speech signal [2]. One such type of speech codec that has proved effective are neural speech codecs, these models are able to learn a mapping from raw audio signals to a compressed latent version and decode. The strength of these models is evident as they can achieve better perceptual quality at low bitrates but have higher latency when compared to other methods such as a traditional source-filter model [3]. Differing approaches to how a model process, transmits, and decodes audio proves to come with tradeoffs that makes deployment on resource-constrained devices in real-world environments an ongoing issue. The ability to develop a neural speech codec that is efficient in terms of compute power, memory, bitrate, and latency proves to be a difficult challenge.

B. Problem Statement

Many modern neural speech codecs learn use an end-to-end learned architecture which consists of a convolutional encoder, residual vector quantization, and a convolutional/ GAN decoder pattern. These models prove to be accurate at reconstructing the original audio input, they require large architectures and higher latency which decrease model efficiency [3]. Hybrid approaches, ones which utilize other models' efficiency in combination with the reconstruction capabilities of neural speech codecs, have proved effective at increasing efficiency in real-world settings [4].

Linear Predictive Coding allows for a low computational load whilst being effective at low bitrates. LPC works by encoding short frames into parameters and decodes by reconstructing the source and filtering it through an estimated vocal tract model. This motivates a hybrid architecture in which a LPC model handles the predictable speech whilst a lightweight neural network can encode the information from the LPC's prediction residuals.

C. Significance

This project investigates if combining LPC with learned residuals from a lightweight neural network can combine the efficiency and quality aspects of the two individual approaches. We propose a hybrid PLC-Neural system in which LPC analysis separates speech into predictive parameters and an excitation residual, the neural network encoder and quantizer

would then compress the residual representation. The system will be evaluated against the LRAC 2.0 baseline across multiple bitrate settings, transparency, robustness to noise and reverb, and intelligibility through a crowdsourced subjective battery.

The central research question is whether explicitly removing predictable spectral structure through LPC allows a neural residual codec to achieve a better quality–complexity tradeoff than the LRAC 2.0 baseline while satisfying the challenge's bitrate, latency, and compute constraints.

II. Related Work

A. LPC

Linear predictive coding (LPC) is an all-pole source-filter model in which speech is modeled as the output. LPC is driven by an excitation signal and generates predictor coefficients which are estimated by minimizing mean-squares prediction error over short [5]. The way this model works is by decomposing the short-time spectrum into a flat excitation spectrum multiplied by a smooth spectral envelope which captures resonances [6]. LPC is primarily used for low-bit-rate speech, especially in the variant known as Codebook Excited Linear Speech (CELP).

B. End-to-End Neural Speech Codecs

There exists multiple end-to-end Neural Speech Codecs that perform all aspects of encoding, transmitting, and decoding. One such example would eb that of SoundStream [7]. SoundStream labels itself as *a novel neural audio codec that can efficiently compress speech, music, and general audio at bitrates normally targeting by speech-tailored codecs*. The method in which the researchers accomplished this was by using an encoder/decoder network and a RVQ which were trained together end-to-end. The researchers demonstrated that the model could operate on bitrates from 3kbps to 18kbps with minimal loss, they also demonstrated the ability to run on a smartphone CPU indicating low latency.
SoundStream shows that end-to-end neural speech codecs can operate at both a low bitrate and low latency, within the parameters of the LRAC 2.0 challenge, and perform in real world setting on devices such as mobile phones.

C. Hybrid Architectures

There exist hybrid approaches that combine the complexity of neural encoder/decoder networks as well as foundational speech models such as LPC. One of the most notable implementations is that of LPCNet [8]. The researchers utilized a WaveRNN variant that combines the linear prediction of LPC with RNN to improve the efficiency of speech synthesis. LCPNet uses LPC to represent the vocal-tract response and trains the neural component to predict the excitation residual. The motivation for this division of labor is that neural architecture can limit compute power as it doesn't need to process the predictable component. The authors conclude that their hybrid approach is acceptable for low bitrate speech coding.
The existing literature indicates that our proposed solution is a defensible approach as both the individual components, and the hybrid methodology, have been tested against similar metrics to those measured in the LRAC 2.0 challenge.

III. Proposed Approaches

We will first reproduce the LRAC 2.0 organizer baseline and establish reference benchmarks across the five (1, 6, 12, 24, 32) bitrates. This will provide us with the comparison point for evaluating whether the proposed LPC-neural hybrid architecture improves the quality-efficiency tradeoff.

The first aspect of implementing our proposed approach will consist of implementing a LPC front-end decomposition. This conceptually means that we will have our input audio waveform, and standard for how traditional LPC works will divide the waveform into short frames. On each frame, we will perform LPC analysis meaning that we will extract the LPC (known) coefficients of the speech, as well as compute the excitation residuals. The LPC coefficients derived in this step represent the predictable spectral structure of the speech whilst the residual information is what gets fed to the neural model. This breakdown allows for the neural model to deal with less data when compared to a compressed version of the entire waveform thus reducing the latency issue found with end-to-end neural model approaches.

The residual will then be passed to a lightweight neural encoder in which a compressed latent representation will be derived. From here, the compressed latent representation will then be quantized before having a lightweight neural decoder reconstruct the residual. After reconstruction, LPC synthesis will reconstruct the final speech.

The planned neural residual codex will use a lightweight causal one-dimensional convolutional architecture that will operate directly on the LPC residual. The encoder will have an initial causal convolution followed by residual down sampling blocks. These blocks will learn local excitation patterns while requiring fewer operations than standard convolutions. The encoder will progressively reduce the temporal resolution and produce a compact latent representation without using future audio samples; this is done to remain within the LRAC 2.0 latency and computational limits.

The latent representation will be compressed through residual vector quantization (RVQ). Quantizer dropout will allow the same encoder and decoder to support all five required bitrate caps by activating less quantizer stages at lower bitrates and additional stages at higher bitrates. A lightweight causal decoder will reconstruct the excitation residual which will then be combined with the LPC parameters via LPC synthesis to reconstruct the speech waveform.

After the architecture has been completed, we will train and evaluate with permitted noise augmentation, use room impulse responses for reverberation, and evaluate whether the LPC decomposition is effective with the degraded audio.

The final model will be compared with the organizer baseline as well as the initial model to see how the proposed architecture compares in both a clean setting as well as with degraded sound.

It is worth noting that although model selection is similar to that of other work such as LPCNet. LPCNet showed that linear prediction can reduce the complexity of neural speech synthesis via removing the predictable response as something the neural network models. What the researchers did was have the network predict the excitation signal from the LPC derived speech features which are combined with the linear prediction to generate the output. Our proposed system differs in both architecture and objective. We will use the LPC as a frond-end decomposition, the LPC parameters represent the predictable spectral structure while the prediction residual is given to a lightweight encoder, quantization stage, and neural decoder. This structure allows for us to evaluate bitrate, quantization, latency, and computational tradeoffs in accordance with the LRAC 2.0 constraints.

IV. System Design

The architecture for the proposed solution can be found in *Figure 1.*

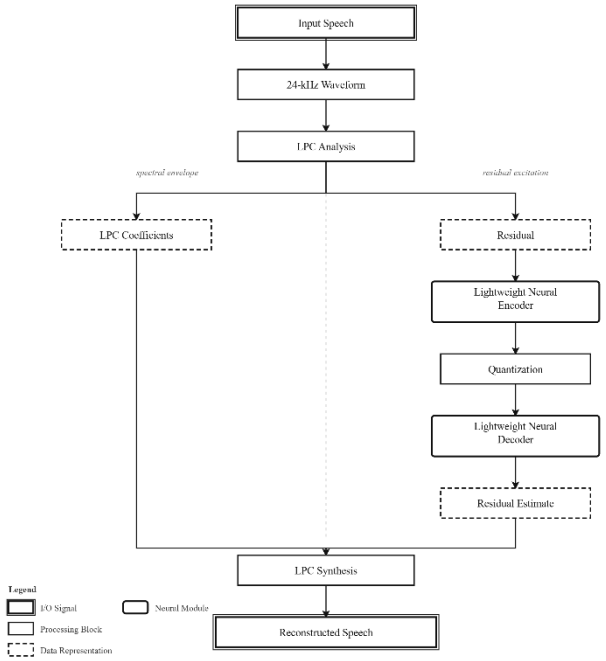


Figure 1: Architecture

For a predictor of order *p*, the predicted sample $\hat{x}[n]$ and the residual e[n] are given by

$$\hat{x}[n] = \sum_{k=1}^{p} a_k \, x[n-k],$$

$$e[n] = x[n] - \hat{x}[n]$$

Where x[n] is the input waveform and a_k denotes the kth predictor coefficient for the current frame. The predictor coefficients describe the short-term spectral envelope, while the redisual contains the portion of the waveform that the predictor does not explain. The neural encoder maps the residual to a lower-rate latent representation, and RVQ represent that latent using codebook indices.

The neural decoder reconstructs an estimate of the residual from the received indices. The synthesis filter then reconstructs the waveform according to

$$\tilde{x}[n] = \tilde{e}[n] + \sum_{k=1}^{p} \tilde{a}_k \, \tilde{x}[n-k]$$

where the equation represents the decoded residual. The filter operates on previously reconstructed samples, so the state has to be handled consistently across frame boundaries. The residual has the same number of samples as the input waveform before neural encoding. The hypothesis is that removing the predictable structure makes the residual easier to compress and a given bitrate and model capacity.

The proposed architecture will process a 24-kHz speech using frame-based LPC analysis, meaning the original speech is broken up into smaller chunks to be processed. The predictor coefficients will represent the short-term spectral envelope, while the prediction residual will be passed to a lightweight neural encoder. The encoder will produce a compressed latent representation that is then discretized via a quantization stage. A neural encoder will then reconstruct the residual signal before being combined with the LPC parameters with a LPC filter to reconstruct the waveform.

Due to the constraints of the LRAC 2.0 challenge, limited lookahead layers will be used to keep the latency of the system low (<30ms). The encoder and decoder will be benchmarked throughout development to measure MFLOPS, parameter count, and the overall complexity of the model. These steps would further simplify the complexity of the neural architecture past removing the predictable structure with the LPC front-end.

It is also worth noting that there are additional constraints laid out by the LRAC 2.0 Challenge, this means that our solution must have: total compute < 700 MFLOPS, total latency < 30ms, must support all bitrate caps and mixture of caps in one inference run, and track 2 requires < 1.5 million parameters.

The proposed codec divides its bit budget between the neural representation of the excitation residual and the side information needed for LPC synthesis. For a target rate of R bits per second and a coding interval of T seconds, the total budget is RT bits per interval. The following table shows the

budget at each target rate using a 20 ms interval as an example. The values represent the total budget before allocating bits to individual codec components.

Target bitrate	Total bits per interval
1 kbps	20
6 kbps	120
12 kbps	240
24 kbps	480
32 kbps	640

The 1 kbps illustrates the tradeoff in the hybrid architecture. If the predictor parameters are sent repeatedly or represented with high precision, then fewer bits are available for the residual. Send the parameters less frequently would allow more room for the residual, but the decoder would have to use the same predictor for a longer segment of speech. This could affect reconstruction during transitions between sounds, where the short-term spectral envelope can change. The predictor update interval and precision used to represent the coefficients need to be considered together with the number of active residual quantizer stages.

The codec also needs to handle changes in bitrate within a continuous stream. When the encoder changes the number of active quainter stages, the decoder must know how many indices to read for the next coding interval. The decoder must also retrain the synthesis filter needed to reconstruct the next samples.

The programming language that will be used is Python, this is chosen due to the available deep learning frameworks and associated libraries. We will utilize PyTorch as our deep-learning framework. For the audio processing, we will utilize torchaudio, NumPy, and SciPy. Visualization of benchmarking and intermediate testing will be done using Matplotlib. Development tools will consist of Git/GitHub, CUDA, and Virginia Tech resources including GLogin for computationally expensive workloads.

The datasets we will be using is the official LRAC 2.0 data provided on the challenge page. The dataset consists of permitted speech, noise, and RIR datasets that total 1200 hours of audio across 12 languages.

For evaluation of our proposed solution, we will evaluate over 4 different criteria. To measure quality, we will utilize the objective quality metrics provided. To measure intelligibility, we will compare with baseline metrics. We will also be looking at resource constraints by measuring bitrate, latency, MLFLOPS, parameter count, and memory cost. We will finally measure robustness by comparing clean speech, noisy speech, reverberant speech, and multilingual speech.

V. Checkpoint Evaluation Plan

A. Component Verification

Before comparing the proposed codec with the organizer baseline, we will test whether the LPC analysis and synthesis stages can reproduce an input waveform when the original predictor coefficients and residual are supplied to the decoder. We will compare the reconstructed waveform with the input and examine the error around fram boundaries. This check is important because the synthesis filter depends on previously reconstructed samples. If the encoder and decoder handle handle the state differently, reconstruction errors can appear even before the residual in compressed. If we establish the reconstruction error with uncompressed inputs, that will give us a reference for evaluating the effect of adding the neural codec.

The next test will replace the original residual with the output of the neural decoder while keeping the LPC coefficients fixed. Comparing the two reconstructed waveforms will show how errors in the decoded residual affect the final speech signal. Once the decoder's representation oif the predictor coefficients is available, we will repeat the test using only the information available to the receiver. This final step will allow us to evaluate both sources of distortion together and measure the total number of bits required to reconstruct the utterance.

B. Initial Baseline Comparison

The first end-to-end experiment will compare the proposed codec with the organizer baseline at one bitrate that both systems can support. Each codec will process the same held-out utterances, and we will record the measure bitrate and reconstruction quality for each system. We will calculate bitrate using the coded output size and the duration of the input. For the proposed codec, that count will include the residual indices and any transmitted information required for LPC synthesis. This is important at the lowest target rate, where the predictor parameters may account for a large portion of available bits.
We will also report algorithmic delay and execution times separately. Algorithmic delay will account for input buffering and any lookahead required to produce the corresponding decoded samples. Execution time will measure how long the encoder and decoder take to process the input on the hardware. Using the same hardware and input clips for both systems will make the runtime comparison easier to interpret. The first comparison will establish a common test procedure that can be repeated at additional bitrates as the configuration become available.

C. Evaluation Under Noise

After establishing a clean-speech comparison, we evaluate the codecs on noisy and reverberant versions of the same utterances. The same noise recordings and room impulse responses will be used for each coded so that differences in the input conditions do not affect the comparison. We will report results of each condition separately to show whether changes

in quality are concentrated in clean speech, noisy speech, or reverberant speech.

Test	What will be Compared	What the test will establish
LPC Reconstruction	Input waveform and waveform reconstructed from the original residual and coefficients	Whether the analysis and synthesis stages handle predictor coefficients and filter state consistently
Residual compression	LPC synthesis using the original residual and synthesis using the decoded residual	How residual compression affects the reconstructed waveform
Baseline comparison	Organizer baseline and codec at a common measured rate	How the compete systems compare in reconstruction quality and resource use
Robustness comparison	Clean-trained and augmentation-trained hybrid models on the same test inputs	Whether augmentation changes performance on clean, noisy, or reverberant speech

VI. Appendix

The roles of each team member are as follows:

Name	Primary Role(s)	Responsibilities
Andrew	Team Lead, System Integration, and LRAC Compliance	Coordinate project milestones and team integration; maintain the main code repository; reproduce and integrate the LRAC baseline; verify bitrate, latency, and computational constraints; integrate LPC, neural codec, and evaluation components into the final system; coordinate proposal, presentations, and final report.
Matt	Data Pipeline, LPC Processing, and Robustness Augmentation	Prepare the LRAC 2.0 datasets using the official pipeline; implement frame-based LPC analysis and synthesis; generate LPC coefficients and excitation residuals; implement permitted noise and room-impulse-response augmentation; maintain clean, noisy, reverberant, and multilingual evaluation subsets.
Wayne	Neural Codec Development, Training, and Evaluation	Develop and train the lightweight neural residual encoder, quantization stage, and decoder; conduct baseline and proposed-model experiments; evaluate reconstruction quality and intelligibility; profile model size, MFLOPS, and latency; produce plots and experimental results for checkpoints and the final report.

Although each team member has roles that they are responsible for, all members will contribute to each stage. The primary roles designation indicates which team member "owns" that step of the project.

The schedule is as follows:

Date / Period	Planned Work and Deliverables
September 22	**Project Proposal Due.** Finalize research question, LPC-neural architecture, related work, task assignments, schedule, and project webpage.
September 23 – October 3	Set up development environment and repository. Register for LRAC 2.0, obtain permitted datasets, run the official data-

	preparation pipeline, reproduce basic organizer baseline inference, and implement initial LPC analysis/synthesis.
October 4 – October 13	Reproduce and profile the organizer baseline. Measure bitrate, reconstruction quality, latency, computational complexity, and model size. Validate LPC decomposition and residual reconstruction on a development subset.
October 14 – October 20	Integrate the initial lightweight neural residual encoder, quantization stage, and decoder. Perform preliminary baseline vs. LPC-neural comparisons. Prepare architecture and initial results. **Checkpoint I due October 20.**
October 21 – October 31	Refine the LPC-neural architecture and required bitrate support. Implement permitted noise and reverberation augmentation. Begin clean vs. degraded-speech experiments.
November 1 – November 10	Conduct ablation experiments comparing the organizer baseline, LPC-neural codec, and robustness-enhanced LPC-neural codec. Evaluate multilingual performance, latency, MFLOPS, and quality-compute tradeoffs. **Checkpoint II and 15-minute presentation due November 10.**
November 11 – November 21	Optimize the architecture based on checkpoint results. Investigate alternative quantization, model compression, or knowledge distillation if time permits. Complete major experiments and constraint-compliance testing.
November 22 – November 30	Freeze the final model configuration. Complete rate-quality, robustness, multilingual, and efficiency experiments. Prepare final figures, tables, and LRAC submission materials.
December 1	Process the LRAC blind-test data and submit challenge materials, if participating. Begin final project presentation period.
December 1, 3, and 8	**Final Project Presentations.** Present final architecture, experimental results, ablation study, LRAC compliance, limitations, and conclusions on assigned date.
December 2 – December 10	Incorporate presentation feedback, complete final analysis and discussion, clean repository/project webpage, and finalize the paper.
December 11, 8:00 PM	**Final Report Due.** Submit final report with methodology, implementation, experiments, results, analysis, limitations, and future work.

VII. Contributions

A. *Andrew Thibodeau*

Andrew coordinated the teams work through the first project checkpoint and maintained the shared repository and integrated system. He led reproduction and profiling of the LRAC 2.0 organizer baseline which included verification of its inference pipeline and measured bitrate, latency, computational complexity, parameter count, and reconstruction performance. Andrew also developed the initial system architecture and compliance testing workflow used to determine whether the proposed system satisfies the LRAC 2.0 requirements.

B. *Matthew Hearne*

Matthew prepared the LRAC 2.0 dataset and established the initial training, development, and evaluation subsets used by the team. He implemented the LPC analysis and synthesis pipeline, including speech framing, predictor coefficient, prediction residual generation, and waveform reconstruction. Matthew validated the LPC implementation by comparing reconstructed waveforms with the original input and examining the residuals characteristics.

C. *Wayne Cleary*

Wayne designed and implemented the initial lightweight neural residual codec. His work included the causal convolutional encoder, latent bottleneck, RVG stage, and the neural decoder. He configured the initial reconstruction losses and training procedure, trained the model on LPC prediction residuals, and examined whether the model could reconstruct excitation information across the initial bitrate limits.

VIII. References

[1] Cisco, "LRAC 2.0 Challenge," *Low-Resource Audio Codec Challenge*,2026.[Online].Available: https://crowdsourcing.cisco.com/lrac-challenge/2026/. [Accessed: Sep. 22, 2026].

[2] G. K. Sheferaw, W. Mwangi, M. W. Kimwele, and A. L. Mamuye, "Waveform based speech coding using nonlinear predictive techniques: A systematic review," *International Journal of Speech Technology*, vol. 26, pp. 1031–1059, 2023, doi: 10.1007/s10772-023-10072-7.

[3] X. Jiang, X. Peng, H. Xue, Y. Zhang, and Y. Lu, "Latent-domain predictive neural speech coding," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 31, pp. 2111–2123, 2023, doi: 10.1109/TASLP.2023.3277693.

[4] M. Kim and J. Skoglund, "Neural speech and audio coding: Modern AI technology meets traditional codecs," *IEEE Signal Processing Magazine*, vol. 41, no. 6, pp. 85–93, Nov. 2024, doi: 10.1109/MSP.2024.3444318.

[5] B. S. Atal and S. L. Hanauer, "Speech analysis and synthesis by linear prediction of the speech wave," *The Journal of the Acoustical Society of America*, vol. 50, no. 2B, pp. 637–655, 1971, doi: 10.1121/1.1912679.

[6] J. O. Smith III, "Linear predictive coding of speech," in *Physical Audio Signal Processing*. Stanford, CA, USA: Stanford University, 2010. [Online]. Available: https://ccrma.stanford.edu/~jos/pasp/Linear_Predictive_Coding_Speech.html. [Accessed: Sep. 22, 2026].

[7] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi, "SoundStream: An end-to-end neural audio codec," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 30, pp. 495–507, 2022, doi: 10.1109/TASLP.2021.3129994.

[8] J.-M. Valin and J. Skoglund, "LPCNet: Improving neural speech synthesis through linear prediction," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Brighton, U.K., 2019, pp. 5891–5895, doi: 10.1109/ICASSP.2019.8682804.